CC3092 — Deep Learning y Sistemas Inteligentes

Hoja de Trabajo #2: Transformers y Mecanismos de Atención

Ernesto Ascencio
Carné 23009
git clone https://github.com/AscencioSIUU/deepLearning.git
Carpeta *labs/hdb-transformers*

1. De las RNN a la atención

En un seq2seq con RNN el codificador comprime toda la entrada en un único vector de contexto de tamaño fijo, que es un **cuello de botella**: la calidad cae con la longitud de la oración y el gradiente debe recorrer T pasos (desvanecimiento). **Bahdanau** et al. (2014) permite que el decodificador consulte todos los estados del codificador: $e_{ij} = v^T \tanh(W_s s_{i-1} + W_h h_j)$ (atención **aditiva**, una MLP), $\alpha_{ij} = \mathrm{softmax}_j(e_{ij})$, $c_i = \Sigma_j \alpha_{ij} h_j$. **Luong** et al. (2015) usa scores **multiplicativos** ($s^T h$ o $s^T W h$): equivalentes en calidad pero mucho más baratos, porque se reducen a un producto matricial. El Transformer generaliza esto con **Query/Key/Value**: la *query* de un token pregunta, las *keys* anuncian qué contiene cada token, y los *values* son lo que se lee; el resultado es un promedio de V ponderado por la compatibilidad $q \cdot k$. La atención deja de ser un complemento de una RNN y pasa a ser la única forma de mezclar información entre posiciones.

2. Arquitectura Transformer

A) **Scaled dot-product** (Vaswani et al., 2017): $\mathrm{Attention}(Q,K,V) = \mathrm{softmax}(QK^T/\sqrt{d_k})\,V$. Si las componentes de q y k son independientes con media 0 y varianza 1, $q \cdot k$ tiene varianza d_k; sin la división el softmax se satura y los gradientes se anulan. **Multi-head**: h cabezas con proyecciones propias de dimensión $d_k = d_{model}/h$ atienden en paralelo y se concatenan con W_O; permiten que cada cabeza se especialice en un tipo de relación con el mismo costo total.

B) **Codificación posicional.** La atención es invariante a permutaciones, por lo que hay que inyectar el orden. *Sinusoidal*: $PE(pos,2i)=\sin(pos/10000^{2i/d})$, $PE(pos,2i+1)=\cos(\cdot)$, sumada al embedding; el coseno entre posiciones decae con la distancia (fig. del notebook) y permite extrapolar de forma teórica. *RoPE* (Su et al., 2021): rota cada par de dimensiones de q y k por $pos \cdot \theta_i$, de modo que $q_m \cdot k_n$ depende solo de $m-n$ (verificado numéricamente: −6.36248 para *(3,1)*, *(10,8)* y *(103,101)*). Es posicional-relativa, no añade parámetros y es el estándar en LLM modernos.

C) **Bloque.** Residual + LayerNorm alrededor de la atención y de una FFN. *Post-LN* (original): $LN(x+F(x))$; *Pre-LN*: $x+F(LN(x))$. Pre-LN (Xiong et al., 2020) conserva un camino identidad limpio, tiene gradientes bien escalados al inicio y entrena sin *warm-up* delicado; Post-LN suele rendir algo mejor si converge pero es inestable en modelos profundos.

D) **Complejidad.** Autoatención: $O(T^2 \cdot d)$ tiempo y $O(T^2)$ memoria por cabeza, pero profundidad secuencial $O(1)$ y camino máximo entre dos tokens $O(1)$; una RNN: $O(T \cdot d^2)$ con profundidad secuencial $O(T)$ y camino $O(T)$. Alternativas: **FlashAttention** (Dao et al., 2022; exacta; hace *tiling* en SRAM y no materializa la matriz $T \times T$ en HBM, memoria $O(T)$ y mucho más rápida por menor tráfico de memoria), atención dispersa/ventana (Longformer, Beltagy et al., 2020; BigBird, Zaheer et al., 2020), aproximaciones lineales (Performer, Choromanski et al., 2020) y modelos de estado (Mamba, Gu y Dao, 2023).

E) **APIs usadas** (§1 del notebook): *F.scaled_dot_product_attention* (kernel fusionado; admite *is_causal*), *nn.MultiheadAttention* (*need_weights=True, average_attn_weights=False* devuelve pesos por cabeza; *attn_mask* aditiva o booleana, *key_padding_mask*), *nn.Transformer.generate_square_subsequent_mask* (máscara aditiva con *-inf* sobre la diagonal), y en Hugging Face *AutoModel.from_pretrained(..., attn_implementation="eager")* con *output_attentions=True* (la implementación SDPA no devuelve pesos).

3. Procedimiento matemático manual

Con $d_k=4$, $Q=[[1,0,1,0],[0,2,0,2],[1,1,0,0]]$, $K=[[1,0,0,1],[0,1,1,0],[1,1,1,1]]$, $V=\mathrm{diag}(1,2,3)$:

A) $QK^T = [[1,1,2],[2,2,4],[1,1,2]] \rightarrow$ dividir por $\sqrt{4}=2$: $[[.5,.5,1],[1,1,2],[.5,.5,1]]$.

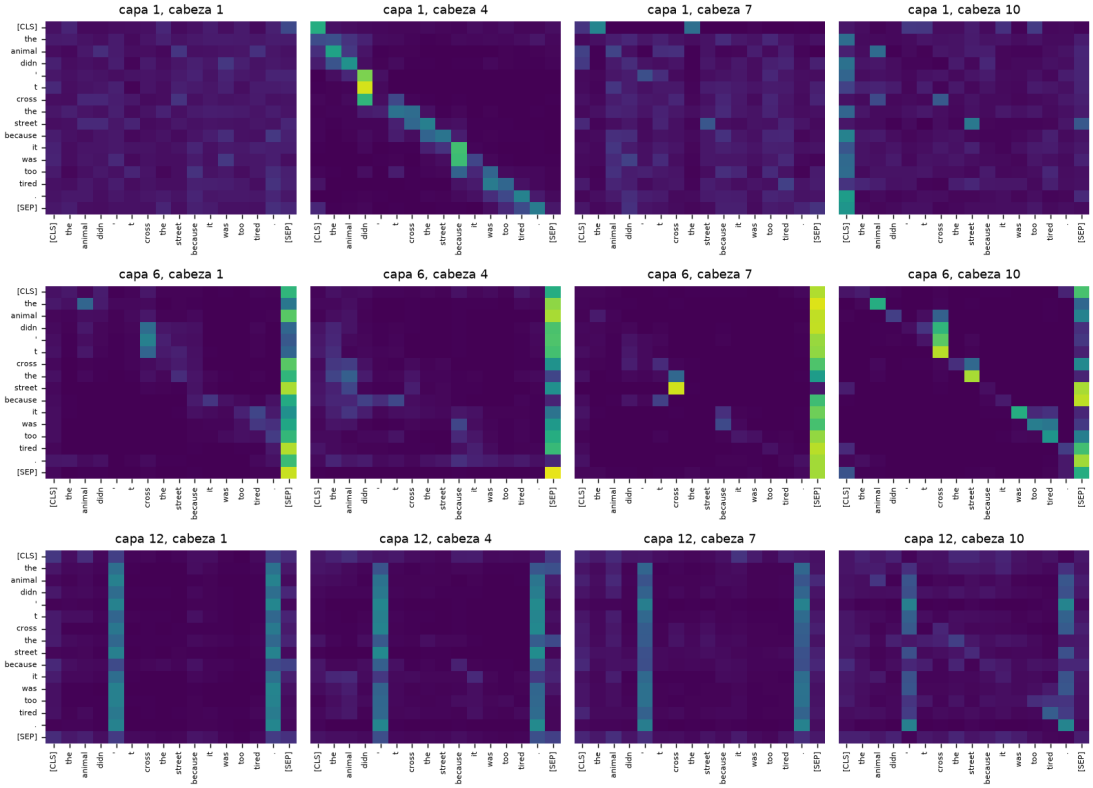
B) Softmax por fila: fila 1 = $e^{.5},e^{.5},e^{1} / (2e^{.5}+e) = [0.2741, 0.2741, 0.4519]$; fila 2 = $[0.2119, 0.2119, 0.5761]$.

C) Salida $A \cdot V$: fila 1 = $0.2741 \cdot [1,0,0]+0.2741 \cdot [0,2,0]+0.4519 \cdot [0,0,3] = [0.2741, 0.5481, 1.3556]$.

D) Con máscara causal la fila 1 queda $[1,0,0]$, la fila 2 $[.5,.5,0]$ y la fila 3 igual que antes.

El notebook comprueba que coincide con *F.scaled_dot_product_attention* y que *nn.MultiheadAttention* reproduce el cálculo manual por cabeza (a partir de *in_proj_weight*) con tolerancia 10^{-5}.

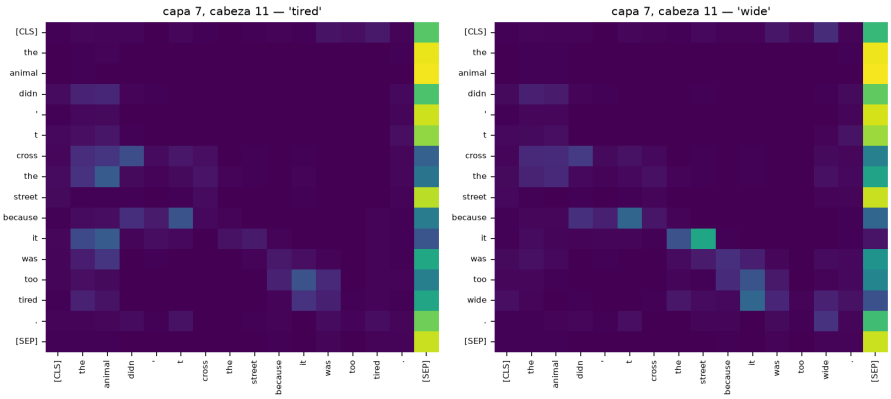
4. Experimentos en BERT y GPT-2



BERT: capas 1, 6 y 12 (filas), cabezas 1, 4, 7, 10 (columnas), sobre "The animal didn't cross the street because it was too tired."

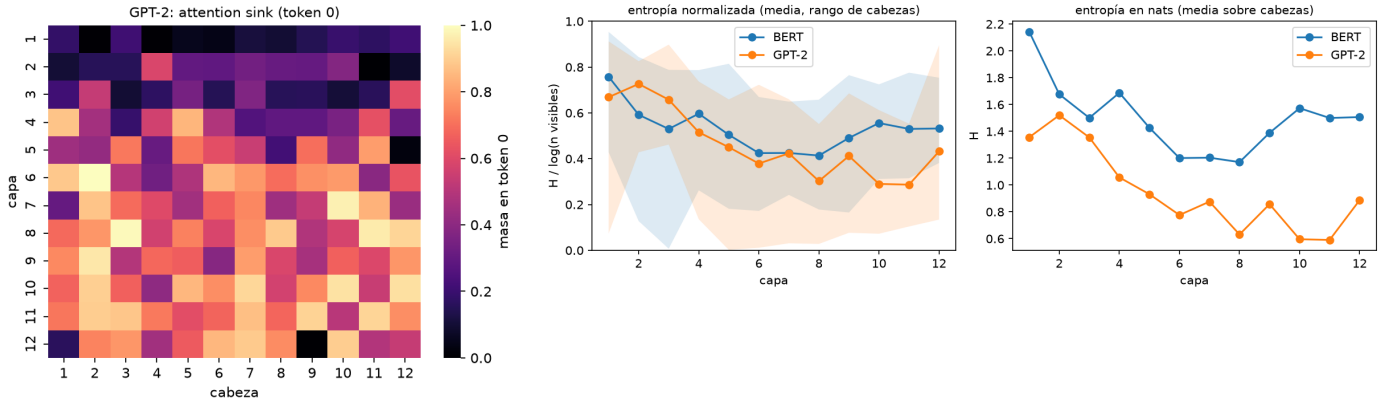
A) **Patrones en BERT.** La atención es bidireccional (matriz llena). En las capas 2–10, 48–63% de la masa media cae en *[CLS]*/*[SEP]* (la capa 6 llega a 0.63), tokens de "reposo" para cabezas sin función útil en esa posición; en las capas 1 y 11–12 solo 9–21%. Hay cabezas puramente posicionales: en la capa 4 una cabeza pone 57% de su masa en el token previo y en la capa 3 una pone 93% en el siguiente. Estos patrones coinciden con los reportados por Clark et al. (2019).

B) **Correferencia** ("it" en *…too tired* vs *…too wide*). Para cada cabeza, $r = A[it \rightarrow animal]/(A[it \rightarrow animal] + A[it \rightarrow street])$; se consideran solo cabezas con ≥5% de masa sobre los dos candidatos (61 de 144), porque r es ruido si no miran a ninguno. En promedio r pasa de 0.712 (*tired*) a 0.531 (*wide*), es decir, el contexto sí desplaza la atención hacia *animal*, pero solo 14 de 61 cabezas (23%) resuelven bien ambos casos, **lo esperable por azar (~25%)**, y se concentran en las capas 7–12. Hay cabezas concretas con comportamiento limpio: capa 7, cabeza 11 da *it→animal*=0.28, *it→street*=0.07 con *tired* y 0.01 / 0.59 con *wide*. Pero elegir la mejor de 61 cabezas sobre un único par de oraciones es selección (comparaciones múltiples), así que no se concluye que esa cabeza "haga correferencia".



BERT: cabeza 11 de la capa 7 en las dos oraciones (izq. tired, der. wide).

C) **GPT-2 causal.** En las 144 cabezas la masa sobre posiciones futuras es exactamente 0 y cada fila suma 1 (la matriz es triangular inferior). **Attention sinks**: 80/144 cabezas dejan >50% de su atención en el token 0 (máx. 0.996) y la masa crece con la profundidad (0.12 en la capa 1, ~0.75 en capas 8–11). Es un efecto **posicional y no semántico**: al cambiar la palabra inicial (*The*, *Ban*, *Quant*, ., salto de línea) la masa se mantiene en 0.52–0.53 con las mismas ~80 cabezas, mientras que *The* en la posición 1 recibe solo 0.025. Con la máscara causal el token 0 es el único visible para todos, y como el softmax obliga a repartir masa 1, sirve de vertedero cuando una cabeza no tiene nada que leer (Xiao et al., 2023).



GPT-2: masa de atención sobre el token 0 por capa × cabeza (izq.) y entropía normalizada por capa en BERT y GPT-2 (der.).

D) **Entropía.** Se calcula $H=-\Sigma p \log p$ por fila, normalizada por *log(tokens visibles)* (en GPT-2 la fila *t* ve *t+1*), promediada sobre 6 oraciones. Ambos modelos arrancan con atención difusa (capa 1: BERT 0.76, GPT-2 0.67) y se vuelven más focalizados con la profundidad (BERT mínimo 0.41 en la capa 8; GPT-2 mínimo 0.29 en las capas 10–11, en parte por los sinks), con repunte en la última capa (0.53 y 0.43). Hay cabezas casi deterministas (entropía normalizada 0.007 y 0.002) y otras casi uniformes (0.95 y 0.91): las cabezas son heterogéneas.

5. Discusión

A) **Escalabilidad frente a las RNN.** Las RNN no se paralelizan en el tiempo (profundidad secuencial *O(T)*) y propagan la información en *O(T)* pasos; el Transformer procesa todos los tokens a la vez y conecta cualquier par en un paso, lo que explica que escale con datos, parámetros y cómputo (leyes de escala). El costo es $O(T^2)$ en memoria y en tiempo para el contexto, y en inferencia el *KV cache* crece linealmente con *T*.

B) **Especialización de cabezas.** Los resultados muestran cabezas posicionales (token previo/siguiente), cabezas “de reposo” sobre *[CLS]*/*[SEP]*/token 0 y algunas con sensibilidad sintáctico-semántica (capa 7, cabeza 11); pero también redundancia: muchas cabezas se pueden podar con poca pérdida (Michel et al., 2019; Voita et al., 2019), y la mayor parte del comportamiento “interpretable” se concentra en pocas.

C) **¿Los pesos de atención explican?** Argumentos en contra (Jain y Wallace, 2019): los pesos pueden alterarse sin cambiar la predicción, y la atención no mide la importancia de un token porque el *value* y las capas siguientes también transforman la señal. Nuestros resultados lo ilustran: los *sinks* concentran 50–99% de la atención en un token semánticamente vacío, así que "mucha atención" no equivale a "mucha influencia". A favor (Wiegreffe y Pinter, 2019): depende de qué se entiende por explicación, y como diagnóstico de dónde se **lee** información sirve. Conclusión: es una pista útil, no una explicación causal; para atribuciones fiables hay que complementarla con intervenciones (ablación de cabezas, *attention rollout*, gradientes).

D) **Contextos de 100,000+ tokens.** T^2 es inviable: 10^5 tokens implican 10^{10} pesos por cabeza y capa. Estrategias: (i) cálculo exacto eficiente (**FlashAttention**, **ring attention** (Liu et al., 2023) para repartir la secuencia entre dispositivos); (ii) patrones dispersos, ventana deslizante + sinks (Longformer, BigBird, **StreamingLLM**, Xiao et al., 2023, que retiene los primeros tokens por el fenómeno anterior); (iii) extensión de posiciones (**RoPE scaling**, interpolación de posiciones, YaRN; Peng et al., 2023) para generalizar más allá de la longitud de entrenamiento; (iv) reducir el KV cache (MQA/GQA, cuantización, compresión); (v) arquitecturas de costo lineal (Mamba, atención lineal); (vi) no meter todo en el contexto: recuperación (RAG) y memoria externa.

Referencias

Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural machine translation by jointly learning to align and translate*. arXiv. https://arxiv.org/abs/1409.0473

Beltagy, I., Peters, M. E., & Cohan, A. (2020). *Longformer: The long-document transformer*. arXiv. https://arxiv.org/abs/2004.05150

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., & Weller, A. (2020). *Rethinking attention with Performers*. arXiv. https://arxiv.org/abs/2009.14794

Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). *What does BERT look at? An analysis of BERT's attention*. arXiv. https://arxiv.org/abs/1906.04341

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). *FlashAttention: Fast and memory-efficient exact attention with IO-awareness*. arXiv. https://arxiv.org/abs/2205.14135

Gu, A., & Dao, T. (2023). *Mamba: Linear-time sequence modeling with selective state spaces*. arXiv. https://arxiv.org/abs/2312.00752

Jain, S., & Wallace, B. C. (2019). *Attention is not explanation*. arXiv. https://arxiv.org/abs/1902.10186

Liu, H., Zaharia, M., & Abbeel, P. (2023). *Ring attention with blockwise transformers for near-infinite context*. arXiv. https://arxiv.org/abs/2310.01889

Luong, M.-T., Pham, H., & Manning, C. D. (2015). *Effective approaches to attention-based neural machine translation*. arXiv. https://arxiv.org/abs/1508.04025

Michel, P., Levy, O., & Neubig, G. (2019). *Are sixteen heads really better than one?* arXiv. https://arxiv.org/abs/1905.10650

Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2023). *YaRN: Efficient context window extension of large language models*. arXiv. https://arxiv.org/abs/2309.00071

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). *RoFormer: Enhanced transformer with rotary position embedding*. arXiv. https://arxiv.org/abs/2104.09864

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. En *Advances in neural information processing systems* (Vol. 30). https://arxiv.org/abs/1706.03762

Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*. arXiv. https://arxiv.org/abs/1905.09418

Wiegreffe, S., & Pinter, Y. (2019). *Attention is not not explanation*. arXiv. https://arxiv.org/abs/1908.04626

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2023). *Efficient streaming language models with attention sinks*. arXiv. https://arxiv.org/abs/2309.17453

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T.-Y. (2020). *On layer normalization in the Transformer architecture*. arXiv. https://arxiv.org/abs/2002.04745

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontañón, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2020). *Big Bird: Transformers for longer sequences*. arXiv. https://arxiv.org/abs/2007.14062